50Days-coding-challenge

Day-17[SQL]

Customer Orders Analysis

Name:M NITHIN

Problem Statement

You are working as a **Data Analyst** for an e-commerce company.
Your task is to analyze customer purchases, orders, and payments using SQL JOIN operations.

You are given **3 tables**:

- customers
- orders
- payments

Write SQL queries to answer the business questions below.

Database Schema customers

Column Name	**Data Type**
customer_id	INT (PK)
customer_name	VARCHAR
city	VARCHAR

orders

Column Name	**Data Type**
order_id	INT (PK)

customer_id INT

order_date DATE **Column**

Name Data Type

amount DECIMAL

payments Column Name

Data Type payment_id

INT (PK) order_id INT

payment_status VARCHAR

Sample Data

```
CREATE TABLE customers (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100),     city
VARCHAR(50)
);

INSERT INTO customers VALUES
(1, 'Amit', 'Bangalore'),
(2, 'Sneha', 'Mumbai'),
(3, 'Rahul', 'Delhi'),
(4, 'Priya', 'Chennai');


CREATE TABLE orders (
order_id INT PRIMARY KEY,
```

```
customer_id INT,     order_date
DATE,

   amount DECIMAL(10,2)
);

INSERT INTO orders VALUES
(101, 1, '2024-01-10', 500),
(102, 1, '2024-02-15', 700),
(103, 2, '2024-03-01', 300),
(104, 5, '2024-03-05', 900);


CREATE TABLE payments (
payment_id INT PRIMARY KEY,
order_id INT,     payment_status
VARCHAR(20)
);

INSERT INTO payments VALUES
(1, 101, 'Completed'), (2,
102, 'Pending'),
(3, 103, 'Completed');
```

Tasks

Task 1: Customer Orders Write

a query to display:

- customer_name
- order_id
- amount

Include only customers who placed orders.

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Open in a new tab

```sql
INSERT INTO payments VALUES
(1, 101, 'Completed'),
(2, 102, 'Pending'),
(3, 103, 'Completed');

SELECT c.customer_name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	customer_name character varying (100)	order_id integer	amount numeric (10,2)
1	Amit	101	500.00
2	Amit	102	700.00
3	Sneha	103	300.00

Total rows: 3 Query complete 00:00:00.155 CRLF Ln 34, Col 1

Task 2: All Customers

Write a query to display:

- all customers
- their order_id (if any)

Customers without orders should still appear.

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
(3, 103, 'Completed');

SELECT c.customer_name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;

SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	customer_name character varying (100)	order_id integer
1	Amit	101
2	Amit	102
3	Sneha	103
4	Priya	[null]
5	Rahul	[null]

Task 3: Invalid Orders Write

a query to find:

- orders that do NOT have a matching customer

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;

SELECT o.*
FROM orders o
LEFT JOIN customers c
ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	order_id [PK] integer	customer_id integer	order_date date	amount numeric (10,2)
1	104	5	2024-03-05	900.00

Total rows: 1 | Query complete 00:00:00.753 | CRLF | Ln 39, Col 1

Task 4: Order Payment Status

Write a query to display:

- customer_name
- order_id
- payment_status

Include all orders, even if payment is missing.

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
SELECT c.customer_name, o.order_id, p.payment_status
FROM orders o
LEFT JOIN customers c
ON o.customer_id = c.customer_id
LEFT JOIN payments p
ON o.order_id = p.order_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	customer_name character varying (100)	order_id integer	payment_status character varying (20)
1	Amit	101	Completed
2	Amit	102	Pending
3	Sneha	103	Completed
4	[null]	104	[null]

Total rows: 4 Query complete 00:00:00.212 CRLF Ln 50, Col 28

Task 5: Customers Without Orders

Find customers who have **never placed an order**.

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
ON o.customer_id = c.customer_id
LEFT JOIN payments p
ON o.order_id = p.order_id;


SELECT c.customer_name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	customer_name character varying (100)
1	Priya
2	Rahul

Total rows: 2 | Query complete 00:00:00.270 | CRLF | Ln 57, Col 26

Task 6: Orders Without Payment

Find all orders that **do not have a payment record**.

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

SELECT o.order_id, o.amount
FROM orders o
LEFT JOIN payments p
ON o.order_id = p.order_id
WHERE p.payment_id IS NULL;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	order_id [PK] integer	amount numeric (10,2)
1	104	900.00

Total rows: 1 | Query complete 00:00:01.205 | CRLF | Ln 59, Col 1

Task 7: Total Spending

Write a query to calculate:

- total amount spent by each customer

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```sql
SELECT o.order_id, o.amount
FROM orders o
LEFT JOIN payments p
ON o.order_id = p.order_id
WHERE p.payment_id IS NULL;

SELECT c.customer_name, SUM(o.amount) AS total_spent
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	customer_name character varying (100)	total_spent numeric
1	Sneha	300.00
2	Amit	1200.00

Total rows: 2 | Query complete 00:00:00.193 | CRLF | Ln 69, Col 26

Task 8: Fully Paid Customers

Find customers whose **all orders are marked as 'Completed'**.

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
GROUP BY c.customer_name;

SELECT c.customer_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments p ON o.order_id = p.order_id
GROUP BY c.customer_name
HAVING COUNT(o.order_id) =
       SUM(CASE WHEN p.payment_status = 'Completed' THEN 1 ELSE 0 END);
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 Page No: 1 of 1

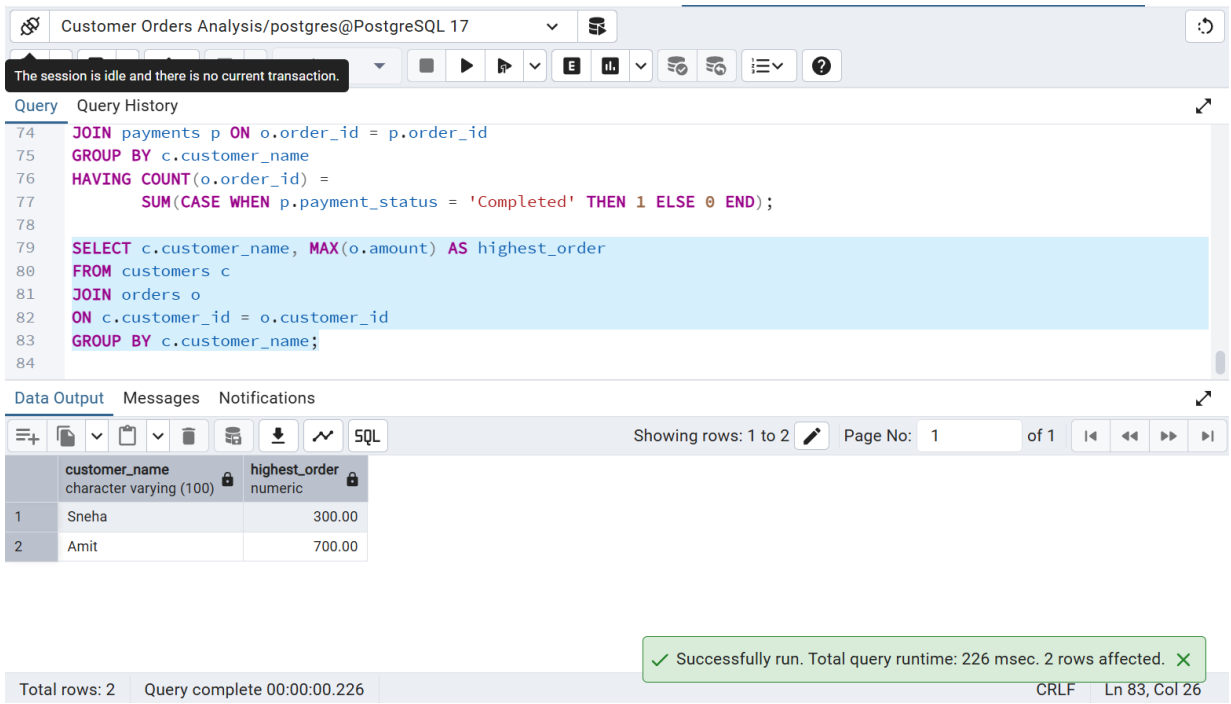
	customer_name character varying (100)
1	Sneha

Total rows: 1 | Query complete 00:00:00.206 | CRLF | Ln 79, Col 5

Task 9: Highest Order Per Customer

Display:

- customer_name
- highest order amount



Task 10: Top Customers

Find **top 2 customers based on total spending**.

Rules

- Use appropriate JOIN types:
  - INNER JOIN
  - LEFT JOIN
  - RIGHT JOIN
- Avoid subqueries unless necessary
- Use GROUP BY where required
- Write clean and readable SQL

Customer Orders Analysis/postgres@PostgreSQL 17

No limit

Query | Query History

```
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;

SELECT c.customer_name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_spent DESC
LIMIT 2;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1

	customer_name character varying (100)	total_spent numeric
1	Amit	1200.00
2	Sneha	300.00

Total rows: 2 | Query complete 00:00:00.213